第2章运行说明

返回正文

正文见 chapter.html。本章复用第 1 章贯穿程序和其编译产物，不新增 Java 源码或空 Maven 模块。用户已授权执行 P03；本章只观察有界正常程序，不编写测试、断言或压力负载。

环境与命令

实际环境为 macOS 27.2、AArch64、Homebrew Java 开发工具包（Java Development Kit，JDK）25.0.5.0.2。先在项目根目录按第 1 章完成编译，选择工具路径：

```
CH02_JDK=/opt/homebrew/Cellar/openjdk@25/25.0.5.0.2/libexec/openjdk.jdk/Contents/Home
```

在允许诊断本进程的终端 A 中启动 60 秒窗口：

```
"$CH02_JDK/bin/java" -Xms32m -Xmx128m -XX:NativeMemoryTracking=detail -cp chapters/ch01-architecture/target/classes org.example.jvminternals.ch01.ProgramJourney chapters/ch01-architecture/src/main/resources/message.txt 60
```

程序打印实际进程号。另一个终端在相同项目根目录和工具选择下，把本轮示例值替换为**当前实际进程号**，例如：

```
CH02_PID=15216
ps -p "$CH02_PID" -o pid,ppid,%cpu,rss,vsz,etime,command
ps -M -p "$CH02_PID"
vmmap -summary "$CH02_PID"
vmmap "$CH02_PID"
"$CH02_JDK/bin/jcmd" "$CH02_PID" Thread.print -l
"$CH02_JDK/bin/jcmd" "$CH02_PID" GC.heap_info
"$CH02_JDK/bin/jcmd" "$CH02_PID" VM.native_memory detail scale=KB
```

15216 是已结束的本轮进程，不可直接照抄为新观察目标。原样实际命令含日志和输出路径，完整保存在 O02 记录。诊断应在窗口内完成；窗口到期自然释放工作线程，堆上限 128 兆二进制字节（mebibyte，MiB），不代表进程总内存限额。

实际证据

O01 保留受限环境中的附加失败；O02 在调整启动权限后成功采集映射、系统 / Java 线程、堆和本地内存跟踪（Native Memory Tracking，NMT）等信息。两次源码与教学输入相同，不混用不同进程的地址。

关注几组不同口径：ps 的驻留与虚拟大小、vmmap 的区域和物理归账、收集器的堆预留 / 提交 / 已用、NMT 的跟踪分类。不是同一时刻的原子采样；附加与 NMT 本身有开销，不能据此比较性能。

页大小和缓存行大小是本机查询结果；架构调用、原子操作与屏障差异另见来源索引。x86-64 只完成资料与实现研究，未声称该架构已经实测。源码、编译身份仍以 CH01 的正式文件为准。